

Resolución de problemas de drivers: NVIDIA y Bluetooth

Compatibilidad de kernel, degradación de driver y
parche DKMS para el ASUS ROG Zephyrus G14 GA403UV

pc-config

Mayo de 2026

Índice

1. Introducción	2
2. Entorno del sistema	2
3. Problema 1: Driver NVIDIA — artefactos y compatibilidad de kernel	2
3.1. Síntoma	2
3.2. Causa	3
3.3. Solución: degradar a nvidia-open 580	3
3.4. Por qué el kernel LTS (6.18) y no el kernel 7.x	3
3.5. Establecer el kernel LTS como arranque predeterminado	3
4. Problema 2: Bluetooth MediaTek MT7922 — fallo en cada arranque	4
4.1. Síntoma	4
4.2. Contexto del driver	4
4.3. Análisis de la causa raíz	4
4.4. La función <code>btmtk_usb_func_query</code>	5
4.5. Solución: módulo DKMS <code>btmtk-fix</code>	6
4.5.1. Estructura del módulo	6
4.5.2. El parche aplicado	6
4.5.3. Comandos de construcción e instalación	7
4.6. Verificación	8
5. Notas de mantenimiento	8
5.1. Si se actualiza el kernel LTS	8
5.2. Si se actualiza <code>linux-firmware</code>	9
5.3. Si se actualiza <code>nvidia-open-dkms</code>	9
5.4. Verificar el estado en cada arranque	9

1. Introducción

Este documento recoge la resolución de dos problemas independientes de drivers que afectaban al **ASUS ROG Zephyrus G14 GA403UV** con **CachyOS** (base Arch Linux) y el entorno **Omarchy** (Hyprland + Wayland):

1. **Driver NVIDIA**: artefactos visuales graves en juegos y incompatibilidad del driver 580 con el kernel 7.x, que obligó a fijar el kernel LTS 6.18 como arranque predeterminado.
2. **Bluetooth MediaTek MT7922**: fallo sistemático en cada arranque con el mensaje `Failed to send wmt func ctrl (-22)`, causado por un cambio de comportamiento en el firmware de febrero de 2026.

Ambos problemas conviven: la solución del Bluetooth requiere el kernel LTS, que a su vez es necesario por la limitación del driver NVIDIA.

2. Entorno del sistema

Componente	Versión / valor
Portátil	ASUS ROG Zephyrus G14 GA403UV
Sistema operativo	CachyOS (base Arch Linux)
Entorno de escritorio	Omarchy (Hyprland + Wayland)
Kernel activo	linux-cachyos-lts 6.18.31-1
iGPU	AMD Radeon RX 780M (HawkPoint) — driver amdgpu
dGPU	NVIDIA GeForce RTX 4060 Laptop (8 GB)
Driver NVIDIA	nvidia-open-dkms 580.95.05-1
Chip Bluetooth	MediaTek MT7922 (USB interno)
Firmware BT	linux-firmware 20260410-1 (build 20260224103448)
Gestor de arranque	Limine 12.2.0

3. Problema 1: Driver NVIDIA — artefactos y compatibilidad de kernel

3.1. Síntoma

Con el driver `nvidia-open-dkms 595.71.05` instalado en el kernel `linux-cachyos` (rama 7.x), el juego *The Last of Us Part I* y otros títulos mostraban **artefactos visuales graves**: corrupciones de textura, píxeles errantes y parpadeos aleatorios que hacían el juego injugable.

3.2. Causa

La versión 595.71.05 de **nvidia-open** introdujo un bug de renderizado conocido que afecta a ciertos juegos bajo Wayland/PRIME Offload en combinación con la iGPU AMD. No se trataba de un problema de configuración sino de un defecto en el propio driver.

3.3. Solución: degradar a nvidia-open 580

Se degradó el driver a la versión **580.95.05-1**, que no presenta el bug:

```
# Descargar la version anterior desde la cache de pacman o un
  mirror
sudo pacman -U nvidia-open-dkms-580.95.05-1-x86_64.pkg.tar.zst \
               nvidia-utils-580.95.05-1-x86_64.pkg.tar.zst \
               lib32-nvidia-utils-580.95.05-1-x86_64.pkg.tar.zst
```

Para evitar que pacman actualice el driver automáticamente, añadir al fichero `/etc/pacman.conf`:

```
IgnorePkg = nvidia-open-dkms nvidia-utils lib32-nvidia-utils
```

3.4. Por qué el kernel LTS (6.18) y no el kernel 7.x

El driver **nvidia-open 580** **no compila** contra el kernel 7.x de CachyOS. El módulo DKMS falla durante la fase de **make** porque la API interna del kernel cambió entre las ramas 6.18 y 7.x en interfaces que el driver 580 utiliza directamente (gestión de memoria de vídeo, interfaz **drm**, etc.).

El intento de construcción produce errores del tipo:

```
# En /var/lib/dkms/nvidia/580.95.05/build/make.log:
error: implicit declaration of function 'drm_...'
error: unknown type name 'struct drm_...'
```

Como resultado, el módulo **nvidia.ko** no se genera y la dGPU queda inaccesible. La única versión de driver compatible con la dGPU RTX 4060 que compila correctamente en CachyOS en el momento de esta guía es la **595.x** (que tiene el bug de renderizado) o la **580.x** (que requiere el kernel LTS 6.18).

La decisión fue: usar **nvidia-open 580 + kernel LTS 6.18**.

3.5. Establecer el kernel LTS como arranque predeterminado

CachyOS usa **Limine** como gestor de arranque. Para que **linux-cachyos-lts** sea la entrada predeterminada en cada arranque, editar `/etc/limine/limine.cfg` y colocar la entrada LTS en primera posición, o bien establecer **DEFAULT_ENTRY** explícitamente:

```
sudo nano /etc/limine/limine.cfg
```

Asegurarse de que la sección del kernel LTS aparece antes que cualquier otra entrada `:OS_ENTRY:`. Limine arranca la primera entrada válida que encuentre. Verificar tras reiniciar:

```
uname -r
# Resultado esperado: 6.18.31-1-cachyos-lts
```

4. Problema 2: Bluetooth MediaTek MT7922 — fallo en cada arranque

4.1. Síntoma

Tras cada arranque, el adaptador Bluetooth MT7922 fallaba al inicializarse. El kernel registraba el siguiente error en `dmesg`:

```
Bluetooth: hci0: Failed to send wmt func ctrl (-22)
```

El código de error `-22` corresponde a `-EINVAL`. El Bluetooth era completamente inoperativo: `bluetoothctl` no mostraba ningún adaptador, y no era posible conectar ningún dispositivo.

4.2. Contexto del driver

El chip MT7922 es gestionado por los módulos `btmtk` y `btusb` del kernel. El fichero de firmware es:

```
/lib/firmware/mediatek/BT_RAM_CODE_MT7922_1_1_hdr.bin.zst
```

La secuencia de arranque del driver es:

1. Descarga del firmware al chip (*firmware download*).
2. Envío del comando **FUNC_CTRL enable**: activa el protocolo Bluetooth en el firmware.
3. Verificación del estado: el driver lee la respuesta para confirmar que el chip está activo.

El fallo ocurría exactamente en el paso 2.

4.3. Análisis de la causa raíz

El protocolo de comunicación entre el driver y el firmware se denomina **WMT** (*Wireless Module Testability*). Cada comando WMT espera una respuesta con una estructura concreta.

Para el comando `FUNC_CTRL` (activar Bluetooth), la respuesta esperada es la estructura `btmtk_hci_wmt_evt_func`, de **9 bytes**:

Campo	Tamaño	Descripción
hci_event_hdr	2 B	Cabecera HCI (código de evento + longitud)
btmtk_wmt_hdr	5 B	Cabecera WMT (dir, op, dlen, flag)
status	2 B	Estado del firmware (0x0404 = activo)

El firmware con fecha de compilación **20260224103448** (febrero de 2026, distribuido con linux-firmware 20260410-1) cambió su comportamiento: en respuesta al comando de activación, devuelve únicamente **7 bytes** — la cabecera completa, pero *sin* los 2 bytes de estado al final.

El driver intenta extraer esos 2 bytes de estado con `skb_pull_data`:

```
case BTMTK_WMT_FUNC_CTRL:
    if (!skb_pull_data(data->evt_skb,
                      sizeof(wmt_evt_func->status))) {
        err = -EINVAL; /* falla: 0 bytes restantes */
        goto err_free_skb;
    }
```

Como el *socket buffer* ya está vacío tras haber consumido la cabecera de 7 bytes, `skb_pull_data` devuelve NULL, la función retorna `-EINVAL`, y el driver registra el error.

Clave: el firmware *sí* procesa el comando y activa el Bluetooth. El cambio es únicamente en el formato de la respuesta. El error es del driver, no del firmware.

4.4. La función `btmtk_usb_func_query`

El driver ya disponía de una función separada para *consultar* el estado del Bluetooth sin activarlo, usando el flag 4 del comando `FUNC_CTRL`:

```
static int btmtk_usb_func_query(struct hci_dev *hdev)
{
    struct btmtk_hci_wmt_params wmt_params;
    int status, err;
    u8 param = 0;

    wmt_params.op = BTMTK_WMT_FUNC_CTRL;
    wmt_params.flag = 4; /* consulta, no activacion */
    wmt_params.dlen = sizeof(param);
    wmt_params.data = &param;
    wmt_params.status = &status;

    err = btmtk_usb_hci_wmt_sync(hdev, &wmt_params);
    if (err < 0) {
        bt_dev_err(hdev, "Failed to query function status (%d)",
                  err);
        return err;
    }
    return status; /* BTMTK_WMT_ON_DONE = 5 si el BT esta activo */
}
```

Esta consulta con flag 4 devuelve la respuesta completa de 9 bytes (incluyendo los bytes de estado) incluso con el firmware nuevo, por lo que funciona correctamente.

4.5. Solución: módulo DKMS btmtk-fix

Como el módulo `btmtk.ko` del kernel no puede modificarse directamente sin recompilar el kernel completo, se creó un módulo DKMS que reemplaza al original. Los módulos en `/updates/dkms/` tienen prioridad automática sobre los del kernel.

4.5.1. Estructura del módulo

```
/usr/src/btmtk-fix-1.0/
  btmtk.c          # copia de drivers/bluetooth/btmtk.c con el
                   parche
  btmtk.h          # cabecera con BTMTK_FIRMWARE_DL_RETRY añadido
  Makefile
  dkms.conf
```

Makefile:

```
obj-m := btmtk.o
KDIR ?= /lib/modules/$(shell uname -r)/build
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

dkms.conf:

```
PACKAGE_NAME="btmtk-fix"
PACKAGE_VERSION="1.0"
BUILT_MODULE_NAME[0]="btmtk"
DEST_MODULE_LOCATION[0]="/updates/dkms"
AUTOINSTALL="yes"
```

4.5.2. El parche aplicado

En `btmtk.h`, se añadió la constante de flag necesaria para el mecanismo de reintento:

```
enum {
    /* ... constantes existentes ... */
    BTMTK_ISOPKT_RUNNING,
    BTMTK_FIRMWARE_DL_RETRY,    /* añadido: indica reintento en
                                curso */
};
```

En `btmtk.c`, dentro de `btmtk_usb_setup`, tras el envío del comando de activación para los chips MT7922/7925/7961, se añadió el bloque de recuperación cuando la respuesta es `-EINVAL`:

```

err = btmtk_usb_hci_wmt_sync(hdev, &wmt_params);
if (err == -EINVAL) {
    /* El firmware nuevo (feb 2026+) no incluye bytes de estado en
     * la respuesta FUNC_CTRL enable. En su lugar, consultar el
     * estado via polling, tolerando -EINVAL mientras el firmware
     * termina de inicializarse tras una carga en frio (hasta ~2s).
     */
    int qstatus;

    err = readx_poll_timeout(btmtk_usb_func_query, hdev, qstatus,
                            qstatus != -EINVAL &&
                            qstatus != BTMTK_WMT_ON_PROGRESS,
                            100000, 3000000);

    if (!err) {
        if (qstatus == BTMTK_WMT_ON_DONE)
            err = 0;
        else
            err = (qstatus < 0) ? qstatus : -EIO;
    }
    bt_dev_info(hdev, "FUNC_CTRL status poll: err=%d qstatus=%d",
                err, qstatus);
}
if (err < 0) {
    if (!test_and_set_bit(BTMTK_FIRMWARE_DL_RETRY, &btmtk_data->
                        flags))
        btmtk_reset_sync(hdev);
    bt_dev_err(hdev, "Failed to send wmt func ctrl (%d)", err);
    return err;
}

```

La macro `readx_poll_timeout` (de `<linux/iopoll.h>`) llama a `btmtk_usb_func_query` repetidamente cada 100 ms durante un máximo de 3 s. La condición de salida es que la respuesta no sea `-EINVAL` (firmware aún inicializándose) ni `BTMTK_WMT_ON_PROGRESS` (carga en curso). Cuando la respuesta es `BTMTK_WMT_ON_DONE` (`=5`), el Bluetooth está activo y el setup continúa con éxito.

4.5.3. Comandos de construcción e instalación

```

# Registrar el modulo en DKMS
sudo dkms add /usr/src/btmtk-fix-1.0

# Compilar para el kernel activo
sudo dkms build btmtk-fix/1.0 -k 6.18.31-1-cachyos-lts

# Instalar (copia el .ko.zst a /updates/dkms/, tiene prioridad)
sudo dkms install btmtk-fix/1.0 -k 6.18.31-1-cachyos-lts

```

Para reconstruir desde cero si es necesario:

```

sudo dkms remove btmtk-fix/1.0 --all

```

```
sudo dkms add /usr/src/btmtk-fix-1.0
sudo dkms build btmtk-fix/1.0 -k 6.18.31-1-cachyos-lts
sudo dkms install btmtk-fix/1.0 -k 6.18.31-1-cachyos-lts
```

El módulo queda instalado en:

```
/lib/modules/6.18.31-1-cachyos-lts/updates/dkms/btmtk.ko.zst
```

El módulo original del kernel queda en:

```
/lib/modules/6.18.31-1-cachyos-lts/kernel/drivers/bluetooth/btmtk.ko.zst
```

4.6. Verificación

Recargar el módulo y comprobar `dmesg`:

```
sudo modprobe -r btusb btmtk
sudo dmesg -C
sudo modprobe btusb
sleep 4
sudo dmesg | grep -iE "hci0.*(HW.SW|FUNC_CTRL|poll|device setup|AOSP|failed)"
```

Salida esperada (fallo **ausente**, setup correcto):

```
Bluetooth: hci0: HW/SW Version: 0x008a008a, Build Time:
20260224103448
Bluetooth: hci0: FUNC_CTRL status poll: err=0 qstatus=5
Bluetooth: hci0: Device setup in 125819 usecs
Bluetooth: hci0: AOSP extensions version v1.00
Bluetooth: hci0: AOSP quality report is supported
```

La línea `FUNC_CTRL status poll: err=0 qstatus=5` confirma que el parche está activo y que el polling detectó correctamente el estado `ON_DONE` del firmware.

5. Notas de mantenimiento

5.1. Si se actualiza el kernel LTS

DKMS recompila automáticamente el módulo `btmtk-fix` para cada nuevo kernel gracias a `AUTOINSTALL=τes` en `dkms.conf`. Si la compilación falla por cambios de API en el kernel, el log de error se encuentra en:

```
/var/lib/dkms/btmtk-fix/1.0/build/make.log
```


5.2. Si se actualiza linux-firmware

El parche es compatible con versiones futuras del firmware. Si el firmware vuelve a incluir los bytes de estado en la respuesta FUNC_CTRL, `err` valdrá 0 (no `-EINVAL`) y el bloque de polling no se ejecutará. El comportamiento es retrocompatible.

5.3. Si se actualiza nvidia-open-dkms

Antes de actualizar el driver NVIDIA, verificar que la nueva versión compila contra el kernel LTS 6.18. Consultar el *changelog* del paquete en AUR o en los foros de CachyOS. Si la nueva versión exige el kernel 7.x, mantener el `IgnorePkg` en `pacman.conf` hasta disponer de un driver compatible.

5.4. Verificar el estado en cada arranque

Para confirmar que ambos componentes funcionan tras un reinicio:

```
# Kernel activo (debe ser 6.18.x-cachyos-lts)
uname -r

# Modulo NVIDIA cargado
lsmod | grep nvidia

# Bluetooth activo (debe mostrar el adaptador)
bluetoothctl show | grep -E "Name|Powered|Discovering"

# Sin errores de BT en dmesg
sudo dmesg | grep -i "bluetooth.*failed"
```